

Finsight

Architecture walkthrough — team Finsight, TBX / BVP Tech Catalyst. Companion to `README.md` .

The model decides *what* to compute. MySQL computes it. The model's words are then checked against the result before anyone sees them. No number the assistant states can originate from the model.

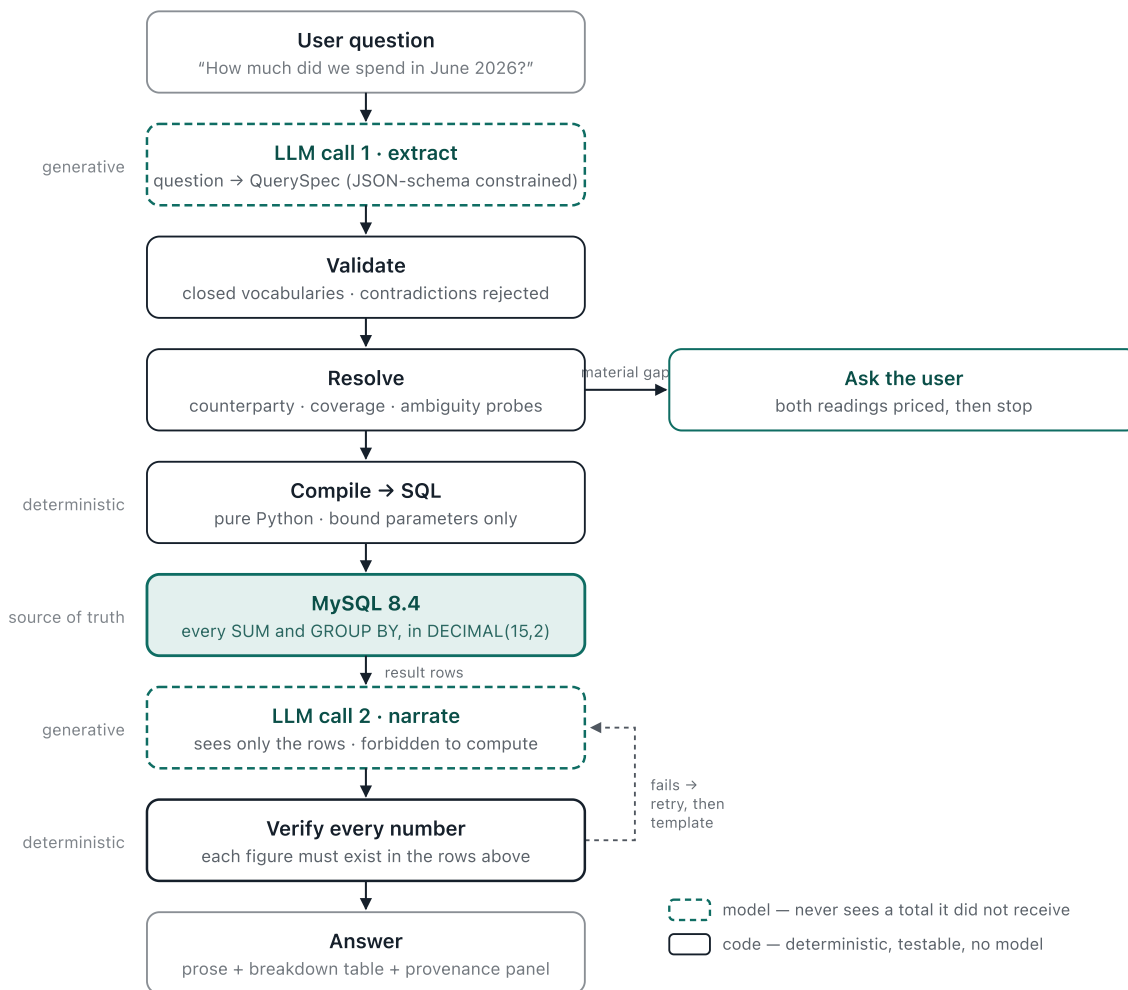
01 The decision everything follows from

The obvious build is text-to-SQL: let the model write a query, run it, show the answer. We deliberately did not. The problem statement rules it out in one sentence — *“aggregate the data correctly **before** handing results to the language model, so the model explains a computed result rather than calculating one itself.”*

	Text-to-SQL	Extract → compile (ours)
Model produces	Free-form SQL	A small typed JSON object
Can emit invalid SQL?	Yes	No — Python writes the SQL
Can invent a number?	Yes	No — arithmetic only in MySQL
Small-model reliability	Poor — generation is hard	Good — extraction is easy
Guardrails attach	After generation, awkwardly	Before compute , on a typed object

That reframe is also why a 7.6 B model is viable: we ask it to do **extraction**, not **generation**. Filling a fixed schema is what small models are good at. The 20 % model-efficiency criterion turns from a constraint into an advantage.

02 The request pipeline



Exactly two model calls per question, and neither one can put a number on screen: the first produces a typed spec, the second may only restate rows MySQL computed.

The ordering has a deliberate consequence for the UI. Rows are emitted at the MySQL step, **before** narration begins — so real numbers appear on screen while the prose is still being written. It reads as fast, and it reinforces visually that the figures came from the database.

2

model calls per question

0

model calls for ambiguity handling

4.1 s

p50 end-to-end · p95 14.2 s

46/46

golden canary, qwen2.5:7b

03 Where wrong answers get stopped

Accuracy and grounding is the largest single criterion at 30 %. Five layers, each cheap, each catching a different failure. Every example below is from this database.

Layer	Catches	Example on this data
1 • Schema pydantic	Invented dimensions, metrics, statuses; contradictory date fields	A metric outside the declared set never compiles — it is rejected and repaired on retry
2 • Sanity pass deterministic	Right shape, wrong question — and filters nobody asked for	"spend" routed to gross_amount : ₹5,46,616 instead of ₹2,49,806, because gross adds credits to debits
3 • Coverage	Periods outside the data	"last month" returns <i>No data for August 2026. Coverage runs 2025-12-03 to 2026-06-24 — never ₹0.00</i>
4 • Ambiguity probe, then decide	Questions with two defensible answers	"the total for this account" — net movement or throughput? Both are priced before either is shown
5 • Provenance post-narration	Any figure in the prose absent from the rows	Verification failure drops confidence to low and replaces the prose with a template built from the rows

Layer 4 is the one worth explaining slowly

Ambiguity is not a property of the question — it is a property of the question *against this data*. "How much did we spend last month" is ambiguous in principle; if the competing readings land on the same number it is not ambiguous in fact, and asking is friction.

So we never ask the model whether something is ambiguous. We detect candidate readings with rules, **run each one as a cheap scalar query**, and decide from the measured gap. Under 1 % we answer silently; up to 10 % we answer and state the assumption; beyond that we ask — showing the real number for each option, so the user recognises their intent instead of parsing jargon. Zero extra model calls.

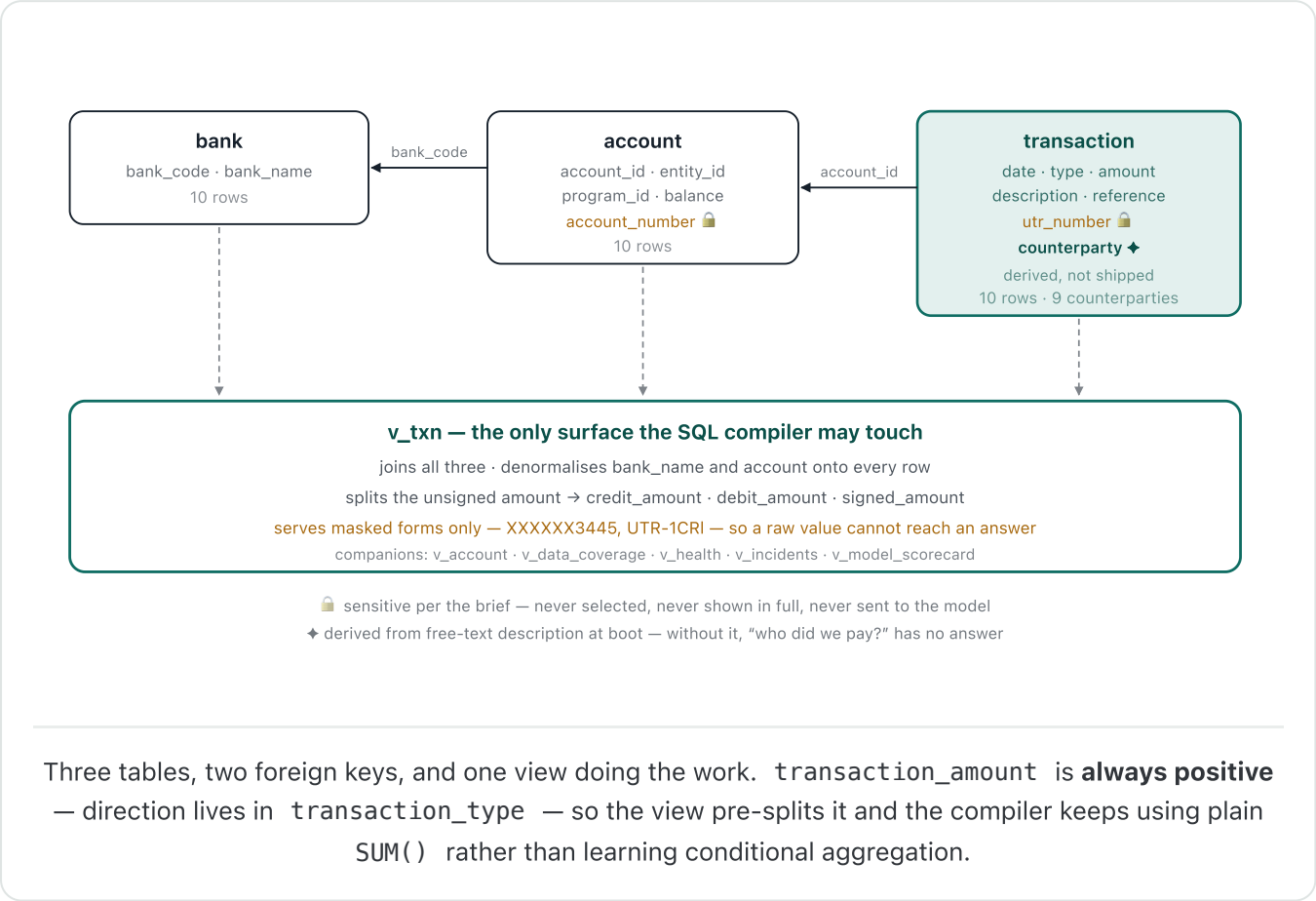
Percentage alone was the wrong test. A fork can sit below the ask threshold in percentage terms and still be worth crores in absolute terms. A second trigger now fires on absolute gap — 0.5 % of total spend, so it scales with whatever dataset is loaded.

The trap that produced a confident, verified, false answer

SUM() over nothing returns **one row** — (NULL, 0) — not zero rows. So every `if not rows` check is blind to it, and the model reads that row as the number zero. Measured, before this was handled: *"The sum of all transactions regarding the account number 30123456789012 is Rs 0."*

Verified, high confidence, green badge — and false. Zero means “these netted to nothing”; the truth was “no such transactions”. The pipeline now collapses that row and says the filters matched nothing, which is not the same as a total of zero.

04 Data model



The row counts are the organizers' sample export. Nothing in the system hardcodes them: coverage, vocabularies and the money columns are introspected into one registry at boot, so swapping in the full export is a load, not a code change.

05 Stack

Layer	Choice	Why this one
Frontend	Next.js 15, App Router	Types generated from the backend schema, so the contract cannot drift
Transport	SSE, browser → FastAPI direct	No Next proxy — proxying SSE is a known buffering trap
Backend	FastAPI + aiomysql	Async pool; pyformat parameters, so no call site changed in the port

05/09/2026, 16:48Architecture — Finsight

Layer	Choice	Why this one
Database	MySQL 8.4 (Docker)	The organizers' own dialect. <code>DECIMAL(15,2)</code> is exact decimal money
Model	<code>qwen2.5:7b-instruct</code> via Ollama	7.6 B — well inside the 20 B ceiling. Local, free to iterate, schema-constrained decoding
Embeddings	<code>nomic-embed-text</code> (137 M)	Semantic entity resolution, off by default
Fallback	<code>claude-haiku-4-5</code>	Same interface — a two-line <code>.env</code> change
Sensitive fields	AES-256-SIV (RFC 5297)	Deterministic, so the column stays searchable and the ciphertext doubles as a stable pseudonym

06 Meeting the constraints

Constraint	How it is met
≤ 20 B parameter LLM	<code>qwen2.5:7b-instruct</code> — 7.6 B , Q4_K_M, 4.7 GB. Runs on a laptop, no API credits spent
≤ 20 M records	Indexed for it: <code>idx_txn_date</code> , <code>idx_txn_account</code> , <code>idx_txn_counterparty</code> , <code>FULLTEXT</code> on description and counterparty. Two things to revisit at that scale are named in the README's limits
Grounded in the provided schema only	Structural, not promised — the compiler emits only columns the profile declares, and the verifier rejects any figure absent from the result rows
Single company, single currency	INR throughout: ₹ with Indian digit grouping (₹1,69,299.00, not ₹169,299.00), timestamps stored UTC and displayed IST
No fabricated figures	The verifier enforces it per answer; the rate is a monitored signal on <code>/ops</code> — currently 97.4 % against a ≥ 95 % threshold

07 Why this survives the real dataset

- **Nothing hardcodes the data.** Statuses, vocabularies, the date coverage and the money columns are all introspected at boot into one registry (`api/registry.py`). No module anywhere else may name a value in the data.
- **Detectors disarm themselves.** They arm from what the schema actually holds, so on a schema without fiscal years or reconciliation statuses the matching questions simply stop

being asked — no code change either way. That is the profile abstraction working, not dead code.

- **The domain lives in one file.** `api/profiles/bank_txn.py` carries every dimension, metric, filter, prompt rule and absent-concept guard. A new schema is a new profile, not a new pipeline.
- **Migrations are idempotent.** Every file is `CREATE ... IF NOT EXISTS / INSERT IGNORE`, applied on every boot, because a migration that runs at every start must never drop anything.
- **The canary recomputes its ground truth.** Expected values are queries, not literals, so the 46-question set stays valid the moment the real export lands.

Companion documents: `README.md` (setup, samples, model choice) · `PLAN.md` (strategy) · `FLOW.md` (interfaces) · `AMBIGUITY.md` (clarification design) · `MYSQL_PORT.md` (the port).